

Inference Methods for KG-Commit

A Survey of Graph Neural Networks, Knowledge Graph Reasoning,
and Downstream Prediction Approaches

KG-Commit Research Project

June 3, 2026

Legend for shaded analysis boxes:

- **Efficiency** — computational and memory cost for training and inference
- **Prior Implementations** — readiness and difficulty of adoption
- **Pros** — advantages specific to KG-Commit
- **Cons** — disadvantages and risks specific to KG-Commit

Contents

1	Introduction	3
1.1	The Inference Landscape	3
1.2	Key Principle	3
1.3	General Design Axes	3
2	Family 1: Shallow Graph Convolutional Networks	4
2.1	What It Captures	4
3	Family 2: Attention-Based Graph Neural Networks	5
3.1	What It Captures	5
4	Family 3: Gated and Sequential Graph Neural Networks	6
4.1	What It Captures	6
5	Family 4: Relational and Heterogeneous Graph Neural Networks	7
5.1	What It Captures	7
6	Family 5: Translational Knowledge Graph Embeddings	8
6.1	What It Captures	8
7	Family 6: Bilinear Knowledge Graph Embeddings	9
7.1	What It Captures	9
8	Family 7: Rotation-Based Knowledge Graph Embeddings	10
8.1	What It Captures	10

9 Family 8: Path-Based Reasoning	11
9.1 What It Captures	11
10 Family 9: Rule Learning and Neural-Symbolic Methods	12
10.1 What It Captures	12
11 Family 10: Temporal Knowledge Graph Inference	13
11.1 What It Captures	13
12 Family 11: Subgraph and Inductive Reasoning	15
12.1 What It Captures	15
13 Family 12: LLM-Augmented Knowledge Graph Inference	16
13.1 What It Captures	16
14 Family 13: Contrastive and Self-Supervised Representations	17
14.1 What It Captures	17
15 Family 14: Graph Transformer Architectures	18
15.1 What It Captures	18
16 Comprehensive Comparison	19
17 Recommended Phased Inference Plan	20
18 Open Research Questions	21

1 Introduction

KG-Commit models repository evolution as an incremental heterogeneous knowledge graph updated with every arriving commit. After the *construction phase* (building the KG from commits, code subgraphs, authors, issues, and temporal nodes), the *inference phase* applies reasoning and prediction algorithms over the populated graph. This document surveys every known inference family applicable to KG-Commit, from classical graph neural networks to knowledge graph embedding methods, path-based reasoning, neural-symbolic approaches, temporal inference, and LLM-augmented methods.

The central inference task is **Just-In-Time defect prediction**: given the KG state at commit c , predict whether c introduces a defect before the tests or reviews reveal it. Secondary tasks include **defect localisation** (which specific file, class, or function?), **link prediction** (will this commit fix issue $\#k$?), **anomaly detection** (does this commit break an established pattern?), and **temporal prediction** (when will the next bug cluster appear in component X ?).

For each inference family, four analysis dimensions are evaluated:

1. **Efficiency** — training time, inference latency, and memory footprint
2. **Prior Implementations and Difficulty** — tool readiness and integration effort
3. **Pros** specific to KG-Commit
4. **Cons** specific to KG-Commit

1.1 The Inference Landscape

KG-Commit Graph

Node types: COMMIT, FILE, CLASS, FUNCTION, AUTHOR, TIME,
INTERVAL, BRANCH, ISSUE, STATEMENT, TOKEN, ...

Edge types: MODIFIES, CALLS, FIXES, CO_CHANGES, DDG_REACHES,
CFG_TRUE, AUTHORED_BY, PART_OF, ...

Inference Families:

GNN-based -- aggregates structural neighbourhoods

KGE-based -- embeds entities and relations as vectors/rotations

Path/Rule-based -- reasons over multi-hop paths and logical rules

Temporal -- exploits commit sequence and timestamps

Subgraph -- inductive reasoning from local subgraph patterns

LLM-augmented -- natural language grounding over KG facts

Self-supervised -- learns from unlabelled graph structure

Graph Transformer-- full self-attention with structural encodings

1.2 Key Principle

The KG-Commit graph is *heterogeneous* (multiple node/edge types), *temporal* (commits arrive in sequence), *incremental* (updated per commit), and *label-scarce* (most commits are unlabelled or clean). Any inference method must handle all four properties well to be practically deployable.

1.3 General Design Axes

Axis	Question	Why it matters
Transductive vs. inductive	Fixed graph vs. new nodes at test time	New repos and projects require inductive methods
Homogeneous vs. heterogeneous	Single type vs. multiple node/edge types	KG-Commit has >15 node types and >30 edge types
Shallow vs. deep	1–2 hops vs. multi-hop reasoning	Defects often span multiple hops
Symbolic vs. neural	Interpretable rules vs. dense vectors	Both wanted: explainability and accuracy
Temporal awareness	Ignores timestamps vs. models commit sequence	Bug patterns have strong temporal locality
Scalability	Linear in graph size vs. quadratic	Full Apache history = 10^7 + nodes
Label efficiency	Supervised vs. self-supervised	JIT defect labels are scarce (<5% positive)

2 Family 1: Shallow Graph Convolutional Networks

2.1 What It Captures

Spectral and spatial graph convolutions aggregate information from k -hop neighbourhoods via learnable weight matrices applied to the normalised adjacency. The canonical model, GCN (Kipf & Welling, 2017), computes:

$$H^{(\ell+1)} = \sigma\left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(\ell)} W^{(\ell)}\right)$$

where $\tilde{A} = A + I$ is the adjacency with self-loops. GraphSAGE (Hamilton et al., 2017) samples a fixed-size neighbourhood and concatenates the aggregated neighbour vector with the central node, enabling inductive generalisation to unseen nodes. ChebNet (Defferrard et al., 2016) uses Chebyshev polynomial filters of order K , approximating multi-hop spectral convolution. SGC (Wu et al., 2019) removes non-linearities between layers, reducing K -layer GCN to a single linear transform of the K -th power of the adjacency.

Model formulation for KG-Commit.

Input node features:

```
COMMIT [la, ld, nf, fix, ns, nd, entropy, ...]
```

```
FUNCTION [loc, cyclomatic, fan_in, fan_out, ...]
```

Aggregation: mean/sum/max over 1-hop neighbours per edge type

Output: per-node embedding -> defect prediction head (MLP + sigmoid)

Baseline: treat all edges as a single type (homogeneous projection)

Efficiency. Training: $O(|\mathcal{E}| \cdot d^2)$ per layer for d -dimensional features and $|\mathcal{E}|$ edges. A 2-layer GCN over the KG-Commit graph with 10^5 nodes and 10^6 edges trains in minutes on a single GPU. Inference: one forward pass per commit subgraph, $O(k \cdot d^2)$ for k neighbours. Mini-batch training via neighbourhood sampling (GraphSAGE) reduces memory to $O(\text{batch} \times k^L \cdot d)$ for L layers. SGC inference is a single sparse matrix

multiply — extremely fast. The most compute-efficient GNN family.

Prior Implementations and Difficulty. **PyTorch Geometric** and **DGL** provide production-ready GCN, GraphSAGE, ChebNet, and SGC implementations. Both support heterogeneous graphs via wrapper modules. **Stellargraph** (TensorFlow-based) provides additional baselines. Integration with KG-Commit: export to a PyG **HeteroData** object (via `py2neo` or `neo4j-arrow`). No graph-type-specific code required for a homogeneous baseline. Difficulty: **very easy**.

Pros. Simplest and fastest GNN baseline; essential for ablation studies. GraphSAGE inductive mode handles new repositories and projects at test time without retraining. SGC’s linear formulation is interpretable — the weight matrix directly shows which node features drive predictions. JITLine (2022) uses GCN over file co-change graphs as a competitive JIT-DP baseline, validating this family for the task.

Cons. Homogeneous GCN ignores the rich edge-type vocabulary of KG-Commit: all edges are treated equally, meaning `MODIFIES` and `CO_CHANGES_WITH` carry equal weight. GraphSAGE with uniform sampling may under-sample rare but critical edge types (e.g., `FIXES` edges to issue nodes). Over-smoothing occurs at ≥ 4 layers, washing out local defect signals. No temporal awareness.

Key papers. GCN (Kipf & Welling, ICLR 2017), GraphSAGE (Hamilton et al., NeurIPS 2017), ChebNet (Defferrard et al., NeurIPS 2016), SGC (Wu et al., ICML 2019).

3 Family 2: Attention-Based Graph Neural Networks

3.1 What It Captures

Graph Attention Networks (GAT, Veličković et al., 2018) learn importance weights over neighbours via a learnable attention mechanism:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}h_i \parallel \mathbf{W}h_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}h_i \parallel \mathbf{W}h_k]))}$$

GATv2 (Brody et al., 2022) fixes expressiveness limitations of GAT by applying the linear transform *after* concatenation. The Heterogeneous Attention Network (HAN, Wang et al., 2019) extends this to heterogeneous graphs via meta-path-based attention: first aggregate within each meta-path, then attend over meta-paths with a semantic-level attention vector.

Model formulation for KG-Commit.

Meta-paths for HAN:

```
COMMIT --MODIFIES--> FILE --MODIFIES<-- COMMIT (co-edit similarity)
COMMIT --AUTHORED_BY--> AUTHOR --AUTHORED_BY<-- COMMIT (co-author)
FUNCTION --CALLS--> FUNCTION --CALLS<-- FUNCTION (call-site coupling)
COMMIT --FIXES--> ISSUE <--MENTIONS-- FUNCTION (issue-function link)
```

Node-level attention: weight neighbours by edge type and feature similarity

Efficiency. GAT: $O(|\mathcal{E}| \cdot d^2 + |\mathcal{E}| \cdot K)$ per layer for K attention heads. In practice 2–3 $\times$ slower than GCN per epoch. GATv2: same asymptotic cost, slightly higher constant. HAN: meta-path materialisation adds a preprocessing step ($O(|\mathcal{E}|)$ per short

meta-path). For KG-Commit with 10^6 edges: HAN preprocessing takes 1–5 minutes; GAT training 10–30 minutes per epoch on GPU.

Prior Implementations and Difficulty. **PyTorch Geometric:** GATConv, GATv2Conv, HANConv. **DGL:** GATConv, HAN. Meta-path definition is a configuration file listing meta-path schemas, which must be manually specified for KG-Commit’s schema (a one-time design task). Meta-path materialisation is trivially computable via Cypher queries on the Neo4j graph. GATv2 should be preferred over GAT for expressiveness. Difficulty: GAT/GATv2 **very easy**; HAN (with meta-path setup) **easy**.

Pros. Attention weights are *interpretable*: which neighbours most influenced a defect prediction can be extracted and shown to developers. Multi-head attention naturally captures multiple roles of a commit (large change AND authored by a low-ownership author). HAN’s meta-path design aligns naturally with the KG-Commit schema and encodes domain knowledge about which structural patterns predict defects.

Cons. HAN is sensitive to the choice of meta-paths; wrong meta-paths hurt performance. Meta-path materialisation can be expensive for long paths in a large KG. Standard GAT is provably no more expressive than GCN for certain graph structures — motivation for GATv2. Attention weights are not globally normalised across the whole graph, limiting their use as a global saliency measure. No relational weight separation (all edge types share the same W).

Key papers. GAT (Veličković et al., ICLR 2018), GATv2 (Brody et al., ICLR 2022), HAN (Wang et al., WWW 2019).

4 Family 3: Gated and Sequential Graph Neural Networks

4.1 What It Captures

The Gated Graph Neural Network (GGNN, Li et al., 2016) replaces the standard GNN update rule with a Gated Recurrent Unit (GRU) applied over T propagation steps:

$$h_v^{(t)} = \text{GRU}(h_v^{(t-1)}, \text{AGG}\{h_u^{(t-1)} : u \in \mathcal{N}(v)\})$$

This allows information to flow over longer paths without over-smoothing for a fixed T . Tree-LSTM (Tai et al., 2015) generalises the LSTM cell to tree structures, applicable to the AST subtree of a function. GGNN is the backbone of **Devign** (Zhou et al., 2019), the most widely cited GNN-based vulnerability detection system, applied directly to CPG subgraphs.

Model formulation for KG-Commit.

GGNN over CPG subgraph (Devign-style):

Initialise: node embedding from token/type features
 Propagate $T=8$ steps: GRU update over outgoing CPG edges
 Readout: gated sum aggregation over all node embeddings
 Output: defect probability (sigmoid)

Tree-LSTM over AST:

Leaf-to-root information flow for syntactic features

Efficiency. GGNN: $O(T \cdot |\mathcal{E}| \cdot d^2)$ for T propagation steps. In Devign, $T = 8$ and $d = 200$ is standard. For a single CPG subgraph (500–2000 nodes), a forward pass takes <10ms on GPU. Batched over 100 commits: <1 second. Training over 10,000 commits: 1–2 hours on GPU. Tree-LSTM: $O(n \cdot d^2)$ per tree for n AST nodes. Both are well within single-GPU memory for function-level graphs.

Prior Implementations and Difficulty. **Devign** reference code (available on GitHub) uses a custom PyTorch GGNN. **DGL: GatedGraphConv.** **PyTorch Geometric: GatedGraphConv.** **Tree-LSTM:** DGL TreeLSTMCell. Devign’s Java port requires generating CPG data via Joern and adapting the graph serialisation. The upstream Devign code is research-grade but straightforward to adapt. Difficulty: GGNN **easy**; full Devign pipeline on Java CPG **moderate**.

Pros. GGNN is the academically canonical choice for code-graph defect detection (Devign, ReVeal, dozens of follow-ups). GRU gating prevents over-smoothing even for $T = 8$ steps, enabling long-range information propagation over the CPG. Tree-LSTM is uniquely suited to the hierarchical AST structure built in the construction phase. Both are directly applicable to the CPG subgraphs produced by Joern in the KG-Commit pipeline.

Cons. Fixed propagation step count T must be tuned; for very deep CPGs T may need to be large (slow). Tree-LSTM is only applicable to the AST subgraph (no cycles), not the full CPG. Sequential GRU recurrence prevents full GPU parallelisation across time steps, making GGNN slower per epoch than GCN. Does not natively handle multiple edge types without type-specific adjacency matrices.

Key papers. GGNN (Li et al., ICLR 2016), Devign (Zhou et al., NeurIPS 2019), Tree-LSTM (Tai et al., ACL 2015), ReVeal (Chakraborty et al., 2022).

5 Family 4: Relational and Heterogeneous Graph Neural Networks

5.1 What It Captures

Relational GCN (R-GCN, Schlichtkrull et al., 2018) uses separate weight matrices per relation (edge type):

$$h_i^{(\ell+1)} = \sigma \left(\sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_r(i)} \frac{1}{c_{i,r}} W_r^{(\ell)} h_j^{(\ell)} + W_0^{(\ell)} h_i^{(\ell)} \right)$$

This directly models the distinct semantics of each edge type in KG-Commit. The Heterogeneous Graph Transformer (HGT, Hu et al., 2020) extends this with a transformer-style attention mechanism across node and edge types, using type-specific linear projections for keys, queries, and values. CompGCN (Vashishth et al., 2020) treats entities and relations as co-embedded objects, enabling composition operations (subtract, multiply, circular correlation) over (h, r, t) triples — bridging GNN and KGE families.

Model formulation for KG-Commit.

R-GCN / HGT over KG-Commit:

Node types : COMMIT, FILE, FUNCTION, AUTHOR, ISSUE, CLASS, ...
Edge types : MODIFIES, CALLS, FIXES, DDG_REACHES, CFG_TRUE,
AUTHORED_BY, CO_CHANGES, HAS_METRIC, ...
Per-relation weight matrix W_r (R-GCN, with basis decomposition)
Per-(src_type, rel, dst_type) transformer head (HGT)
Prediction head: sigmoid over COMMIT node embedding

Efficiency. R-GCN: $O(|\mathcal{R}| \cdot |\mathcal{E}| \cdot d^2)$. With basis decomposition (R-GCN option), reduced to $O(B \cdot d^2 + |\mathcal{R}| \cdot B \cdot d)$ for B basis matrices ($B \ll |\mathcal{R}|$). For KG-Commit with ≈ 30 edge types, basis decomposition is essential. HGT: $O(|\mathcal{E}| \cdot d^2)$ with a higher constant than GCN due to transformer heads. Both are $2\text{--}5\times$ slower than GCN per epoch but dramatically more accurate on heterogeneous KGs.

Prior Implementations and Difficulty. **PyTorch Geometric:** RGCNConv (with basis decomposition), HGTConv, RGATConv, CompGCN. **DGL:** RelGraphConv, HGTConv. Both libraries natively support `HeteroData` objects with typed node/edge features. Integration: export KG-Commit to a PyG `HeteroData` object from Neo4j. R-GCN basis decomposition should be used when $|\mathcal{R}| > 15$. Difficulty: R-GCN **easy**; HGT **easy**; CompGCN **moderate** (requires relation embedding design).

Pros. The **single most appropriate GNN family** for KG-Commit. The heterogeneous edge vocabulary (MODIFIES vs. CALLS vs. FIXES) encodes fundamentally different semantics that R-GCN/HGT treats correctly. HGT’s transformer attention over node types naturally handles the size disparity between FUNCTION nodes (many, small features) and AUTHOR nodes (few, rich historical features). CompGCN’s composition operations encode relational reasoning (“commit by low-ownership author who modified high-churn file”) as a composed entity embedding.

Cons. R-GCN parameter count grows with $|\mathcal{R}|$ (mitigated by basis decomposition). HGT requires type-specific feature projections, adding preprocessing complexity when node feature dimensionalities differ across types. Both require the full heterogeneous graph to be in memory or carefully mini-batched. HGT is harder to interpret than GAT (attention weights span types and are not directly human-readable).

Key papers. R-GCN (Schlichtkrull et al., ESWC 2018), HGT (Hu et al., WWW 2020), CompGCN (Vashishth et al., ICLR 2020).

6 Family 5: Translational Knowledge Graph Embeddings

6.1 What It Captures

Translational KGE methods embed entities h, t and relations r in a low-dimensional vector space such that $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$ for observed triples (h, r, t) . TransE (Bordes et al., 2013) is the canonical model. Variants address its limitations: TransH (Wang et al., 2014) projects entities onto relation-specific hyperplanes, enabling many-to-many relations. TransR (Lin et al., 2015) uses relation-specific projection matrices. TransD (Ji et al., 2015) uses entity-relation-specific dynamic mappings.

Scoring functions.

```

TransE:  $f(h, r, t) = -||h + r - t||$ 
TransH:  $f(h, r, t) = -||h_{\text{perp}} + d_r - t_{\text{perp}}||$ 
TransR:  $f(h, r, t) = -||M_r h + r - M_r t||$ 
Training: margin-based ranking loss with negative sampling
Inference: score all candidate tails  $t'$  given  $(h, r, ?)$ 
KG-Commit triples: (COMMIT, MODIFIES, FILE),
                   (COMMIT, AUTHORED_BY, AUTHOR), ...

```

Efficiency. Training: $O(|\mathcal{T}| \cdot d)$ per epoch for $|\mathcal{T}|$ triples and d -dimensional embeddings (typically $d = 100\text{--}300$). Extremely fast: 10^6 triples train in minutes on CPU. Inference: $O(N \cdot d)$ per query (score all N entities as tail candidates). Total model size: $O((N + R) \cdot d)$ — typically a few MB. The most memory-efficient inference family. Ideal for CPU-only deployment or resource-constrained environments.

Prior Implementations and Difficulty. **PyKEEN** (Fraunhofer, production-ready) implements TransE, TransH, TransR, TransD, and dozens of others in a unified framework with automated hyperparameter optimisation. **OpenKE** (THU, production-ready) provides C++-accelerated training. KG-Commit triple extraction: export a triple file from Neo4j via Cypher in one query. Difficulty: **very easy**.

Pros. Zero GNN infrastructure required. CPU-deployable for online inference after training. The triple-scoring formulation maps directly to link prediction queries: “predict which ISSUE this COMMIT will FIX,” “predict which FUNCTION will be changed next.” Highly scalable: training on the full KG-Commit triple set (10^7 triples) is feasible on a single machine.

Cons. TransE cannot model symmetric or one-to-many relations well (e.g., one AUTHOR modifies many FILES). Purely structural — no code features incorporated. The translation assumption does not capture multi-hop compositionality without explicit path encoding. All KGE models are transductive: embedding new commits at test time requires re-training or a separate inductive wrapper. Lower accuracy than GNNs on feature-rich graphs.

Key papers. TransE (Bordes et al., NeurIPS 2013), TransH (Wang et al., AAAI 2014), TransR (Lin et al., AAAI 2015), PyKEEN (Ali et al., JMLR 2021).

7 Family 6: Bilinear Knowledge Graph Embeddings

7.1 What It Captures

Bilinear KGE models score a triple (h, r, t) via a bilinear interaction. DistMult (Yang et al., 2015) uses a diagonal bilinear form $f = \langle h, r, t \rangle$. ComplEx (Trouillon et al., 2016) lifts to complex space enabling asymmetric relations: $f = \text{Re}(\langle h, r, \bar{t} \rangle)$. Tucker (Balažević et al., 2019) uses Tucker decomposition over a core tensor $\mathcal{W}$: $f = \mathcal{W} \times_1 h \times_2 r \times_3 t$. RESCAL (Nickel et al., 2011) uses a full rank- d bilinear form $f = h^\top M_r t$, the most expressive but most expensive variant.

Scoring functions.

```

DistMult:  $f(h, r, t) = \sum_i h_i * r_i * t_i$ 
ComplEx:  $f(h, r, t) = \text{Re}(\langle h, r, \text{conj}(t) \rangle)$  (complex vectors)

```

TuckER: $f(h,r,t) = W \mathbf{x}_1 h \mathbf{x}_2 r \mathbf{x}_3 t$ (Tucker decomp) Training: binary cross-entropy with negative sampling (DistMult, ComplEx) or multinomial log-loss (TuckER)
--

Efficiency. DistMult: $O(|\mathcal{T}| \cdot d)$ per epoch — the cheapest bilinear model. ComplEx: same asymptotic cost, $2\times$ parameters (real + imaginary). TuckER: $O(|\mathcal{T}| \cdot d_e \cdot d_r)$ where d_e, d_r are entity and relation dimensions — slightly more expensive but achieves SOTA on standard benchmarks. RESCAL: $O(|\mathcal{T}| \cdot d^2)$ — quadratic in d , impractical for $d > 100$. For KG-Commit: DistMult or ComplEx for fast training; TuckER for higher accuracy.

Prior Implementations and Difficulty. **PyKEEN**: all models above with one-line instantiation. **AmpliGraph** (TensorFlow, production-ready) provides DistMult, ComplEx with early stopping and evaluation metrics. **LibKGE** (Broscheit et al.) is a research framework with extensive hyperparameter search. All accept standard triple files. Difficulty: DistMult/ComplEx **very easy**; TuckER **easy**; RESCAL **easy** (but slow for large d).

Pros. ComplEx handles asymmetric relations correctly: AUTHORED_BY is asymmetric (AUTHOR→COMMIT $\neq$ COMMIT→AUTHOR), which TransE cannot model. DistMult achieves competitive results with minimal compute. TuckER achieves SOTA link prediction on Freebase and Wikidata benchmarks. The bilinear form enables matrix-factorisation-style analysis: entity embeddings can be interpreted as latent “roles” in the code change process.

Cons. DistMult cannot model anti-symmetric relations. No structural awareness of graph topology beyond the training triples. Cannot incorporate node feature vectors (code metrics, embeddings) without additional wrapper layers. Bilinear models tend to underperform GNNs when rich node features are available. Transductive limitation applies to all variants.

Key papers. RESCAL (Nickel et al., ICML 2011), DistMult (Yang et al., ICLR 2015), ComplEx (Trouillon et al., ICML 2016), TuckER (Balažević et al., EMNLP 2019).

8 Family 7: Rotation-Based Knowledge Graph Embeddings

8.1 What It Captures

RotatE (Sun et al., 2019) models each relation as a rotation in complex space: $\mathbf{t} = \mathbf{h} \circ \mathbf{r}$, where $\circ$ is element-wise complex multiplication and $|r_k| = 1$. This unified framework models three key relation patterns: symmetry, antisymmetry, and composition ($r_3 = r_1 \circ r_2$). HAKE (Zhang et al., 2020) adds a polar-coordinate modulus component to model hierarchical relations (e.g., FILE $\supset$ CLASS $\supset$ FUNCTION). PairRE (Chao et al., 2021) uses two separate relation vectors (head projection and tail projection), improving multi-hop chain modelling. QuatE (Zhang et al., 2019) generalises to quaternion space (4D rotations).

Scoring functions.

RotatE: $f(h,r,t) = - h \text{ circ } r - t , r_k =1$ (complex)
--

HAKE: $f(h, r, t) = \text{modulus term} + \text{phase term (polar coords)}$ PairRE: $f(h, r, t) = - h \text{ circ } r_H - t \text{ circ } r_T $ QuatE: $f(h, r, t) = (h \text{ otimes } r) \cdot t$ (quaternion product)
--

Efficiency. RotatE: $O(|\mathcal{T}| \cdot d)$ per epoch, same as ComplEx. Self-adversarial negative sampling (RotatE’s key trick) adds a small constant overhead. HAKE: same asymptotic cost, slightly higher due to dual-component scoring. PairRE and QuatE: $2\times$ and $4\times$ parameter count respectively, but similar per-epoch cost. GPU training on 10^7 triples: 1–2 hours. CPU inference: $<1\text{ms}$ per link prediction query.

Prior Implementations and Difficulty. **PyKEEN:** RotatE, HAKE, PairRE, QuatE all available. **RotatE** reference code (Sun et al.) is available on GitHub and is actively maintained. HAKE reference code is also available. Self-adversarial negative sampling is the key hyperparameter (temperature α). Difficulty: RotatE/PairRE via PyKEEN **very easy**; HAKE and QuatE **easy**.

Pros. RotatE is the **current SOTA single-model KGE** on standard benchmarks (FB15k-237, WN18RR). HAKE’s hierarchical modelling directly fits KG-Commit’s containment hierarchy ($\text{REPO} \supset \text{FILE} \supset \text{CLASS} \supset \text{FUNCTION}$). PairRE’s multi-hop composition models relation chains ($\text{COMMIT} \rightarrow \text{FUNCTION} \rightarrow \text{CALLS} \rightarrow \text{BUG}$) that appear naturally in defect propagation. Rotation constraints are geometrically interpretable.

Cons. Transductive: same limitation as all KGE methods. The rotation constraint $|r_k| = 1$ restricts relation expressiveness for non-rotational semantics. Self-adversarial negative sampling has a temperature hyperparameter that requires tuning. QuatE requires $4\times$ embedding dimensions, expensive for large entity sets. No code feature incorporation without a wrapper.

Key papers. RotatE (Sun et al., ICLR 2019), HAKE (Zhang et al., ACL 2020), PairRE (Chao et al., ACL 2021), QuatE (Zhang et al., NeurIPS 2019).

9 Family 8: Path-Based Reasoning

9.1 What It Captures

Path-based methods reason over multi-hop paths in the KG. The Path Ranking Algorithm (PRA, Lao et al., 2011) enumerates relation paths between entity pairs and uses them as features for a logistic regression classifier. DeepPath (Xiong et al., 2017) trains a reinforcement learning agent to walk the KG, learning which paths are most predictive. MINERVA (Das et al., 2018) is a fully end-to-end RL path reasoner that navigates from the query entity to the answer entity. M-Walk (Shen et al., 2018) combines MCTS with LSTM-guided path walking for more stable training.

Path formulation for KG-Commit.

Predictive paths for defect prediction:

COMMIT --MODIFIES--> FILE --HAS_SMELL--> GodClass COMMIT --AUTHORED_BY--> AUTHOR --LOW_OWNERSHIP--> FILE FUNCTION --CALLS--> FUNCTION --MAY_BE_NULL--> Statement COMMIT --CHANGES--> FUNCTION <--BACKWARD_SLICE-- Criterion
--

PRA: enumerate paths up to length $L=3$; use as binary features DeepPath/MINERVA: RL agent navigates from COMMIT to predicted label

Efficiency. PRA: path enumeration is $O(|\mathcal{E}|^L)$ for path length L , tractable for $L \leq 3$ with random walk approximation. Feature extraction and logistic regression: fast. DeepPath/MINERVA: RL training is slow (10^3 – 10^4 episodes per relation), typically hours on GPU. Inference: a single forward pass of the policy network per query $O(L \cdot d)$ — very fast once trained.

Prior Implementations and Difficulty. PRA: **ProPPR** (Cohen et al.) provides a production implementation; random walk approximation is trivially implementable in Python with NetworkX. **DeepPath**: reference PyTorch code on GitHub (Xiong et al.). **MINERVA**: reference code on GitHub (Das et al.), maintained. None are currently integrated into any code analysis pipeline. Integration requires a custom KG-Commit environment wrapper for the RL agent. Difficulty: PRA **moderate**; DeepPath/MINERVA **hard** (RL training instability is a known challenge with sparse rewards).

Pros. Highly **interpretable**: paths like “COMMIT $\rightarrow$ MODIFIES $\rightarrow$ FILE $\rightarrow$ HAS_SMELL $\rightarrow$ GodClass” directly explain why a commit is flagged. PRA paths can be displayed to developers as human-readable explanations. Multi-hop reasoning captures change-propagation patterns not visible at 1-hop depth. MINERVA navigates through the actual KG topology, respecting graph structure and relation semantics.

Cons. PRA path enumeration scales poorly to long paths ($L \geq 4$) in large KGs. RL-based methods suffer from sparse reward signals — defect labels are rare ($<5\%$ of commits), making reward assignment difficult. Path-based methods miss structural patterns not on any single path. M-Walk’s MCTS requires multiple rollouts per training step, amplifying the sparsity problem.

Key papers. PRA (Lao et al., EMNLP 2011), DeepPath (Xiong et al., EMNLP 2017), MINERVA (Das et al., ICLR 2018), M-Walk (Shen et al., NeurIPS 2018).

10 Family 9: Rule Learning and Neural-Symbolic Methods

10.1 What It Captures

Rule learning methods mine logical Horn rules from the KG: e.g., $A(x, y) \wedge B(y, z) \Rightarrow C(x, z)$ with a confidence score. AMIE (Galárraga et al., 2013) mines such rules by exhaustive search with statistical pruning. Neural-LP (Yang et al., 2017) learns differentiable rule weights as a soft version of Datalog. DRUM (Sadeghian et al., 2019) mines rules via bidirectional recurrent networks. RNNLogic (Qu et al., 2021) combines rule generation (RNN) with rule-guided reasoning (EM algorithm). These methods produce *symbolic rules* that can be audited by humans and encoded directly into review checklists.

Rule examples for KG-Commit.

Mined rule 1 (defect predictor):

```

MODIFIES(c, f) AND HAS_SMELL(f, GodClass) AND LOC_DELTA > 50
=> LIKELY_BUGGY(c)

```

Mined rule 2 (incomplete change):

```
MODIFIES(c, f1) AND CO_CHANGES(f1, f2) AND NOT MODIFIES(c, f2)
=> INCOMPLETE_CHANGE(c)
```

Mined rule 3 (ownership risk):

```
AUTHORED_BY(c, a) AND OWNERSHIP(a, f) < 0.1 AND MODIFIES(c, f)
=> HIGH_RISK(c)
```

Efficiency. AMIE: rule mining scales as $O(|\mathcal{T}|^2)$ worst case, but with statistical pruning runs in minutes on 10^6 triples. Neural-LP/DRUM: $O(|\mathcal{T}| \cdot d \cdot L)$ per epoch for path length L — similar to KGE. RNNLogic: EM alternation, 10–50 iterations each $O(|\mathcal{T}| \cdot d)$. After training, rule application is $O(|\mathcal{T}|)$ — very fast. Rule sets are tiny (dozens to hundreds of rules), enabling sub-millisecond inference.

Prior Implementations and Difficulty. **AMIE 3.0:** production-ready Java tool, actively maintained. **Neural-LP:** reference TensorFlow code (Yang et al.). **DRUM:** PyTorch reference code. **RNNLogic:** PyTorch, maintained. For KG-Commit: AMIE can be run directly on the exported triple file; mined rules can be reviewed and seeded into the system as interpretable inference rules. This is a strong approach for building *auditable defect prediction rules*. Difficulty: AMIE **easy**; Neural-LP/DRUM **moderate**; RNNLogic **moderate**.

Pros. Fully interpretable: mined rules are auditable by domain experts. AMIE can discover novel co-change rules, smell-defect correlations, and ownership-risk patterns not encoded a priori in the KG schema. Rules are directly actionable as code review checklists. Neural-LP rules are differentiable, enabling end-to-end training. Symbolic rules are portable across projects without retraining on new code.

Cons. Rule mining requires a sufficient number of positive examples to achieve statistical confidence — scarce with few defect-labelled commits. Mined rules may be spurious correlations (not causal). Neural-LP and DRUM do not scale as well as GNNs on feature-rich graphs. Rules mined from one project may not transfer to another (different coding style, different bug classes). No node feature incorporation in purely symbolic rule mining.

Key papers. AMIE (Galárraga et al., WWW 2013; AMIE 3, VLDB 2020), Neural-LP (Yang et al., NeurIPS 2017), DRUM (Sadeghian et al., NeurIPS 2019), RNNLogic (Qu et al., ICLR 2021).

11 Family 10: Temporal Knowledge Graph Inference

11.1 What It Captures

Temporal KGE methods extend entity and relation embeddings with a time component. TTransE (Leblay & Chekol, 2018) adds a time translation: $\mathbf{h} + \mathbf{r} + \boldsymbol{\tau}(t) \approx \mathbf{tail}$. TNTComplex (Lacroix et al., 2020) uses a 4th-order tensor decomposition over (entity, relation, entity, time). RE-NET (Jin et al., 2020) models relational event sequences with a GRU over temporal subgraphs. TiRGN (Li et al., 2022) adds a recurrent encoder over historical entity patterns combined with a local graph encoder. TANGO (Han et al., 2021) uses Neural ODEs to model continuous temporal evolution of entity embeddings.

Temporal formulation for KG-Commit.

Temporal triples:

(COMMIT_t, MODIFIES, FUNCTION, timestamp_t)

Prediction: given history up to $t-1$, predict if COMMIT_t is buggy

TTransE: $h + r + \text{tau}(t) \approx \text{tail}$ (tau = time embedding lookup)

RE-NET: GRU over sequence of historical commit subgraphs

TANGO: $dh/dt = f(h, t)$ via Neural ODE (continuous evolution)

KG-Commit structural assets: TIME nodes, INTERVAL nodes,
timestamp edges, author commit frequency over time windows

Efficiency. TTransE/TNTComplex: same cost as static counterparts plus time embedding storage $O(T \cdot d)$ for T timestamps. RE-NET: $O(T \cdot |\mathcal{G}_t| \cdot d^2)$ where $|\mathcal{G}_t|$ is the size of the temporal subgraph at time t — moderate. TiRGN: combines GNN and RNN, $2\times$ cost of RE-NET. TANGO: ODE solver adds significant overhead ($\approx 10\times$ RE-NET) but is more expressive for continuous dynamics. For KG-Commit: RE-NET is the practical choice; TANGO is research-grade.

Prior Implementations and Difficulty. **PyKEEN:** TTransE, TNTComplex, ATiSE all available. **RE-NET:** PyTorch reference code on GitHub (Jin et al.), maintained. **TiRGN:** PyTorch reference code (Li et al.). **TANGO:** research code. KG-Commit already has TIME nodes and INTERVAL nodes from the construction phase — these map directly to temporal KGE timestamps. Integration requires attaching timestamps to commit triples in the exported file. Difficulty: TTransE/TNTComplex **easy**; RE-NET **moderate**; TiRGN **moderate**; TANGO **hard**.

Pros. The most natural fit for KG-Commit’s commit-sequence structure. Bug patterns have strong temporal locality (bug-introducing commits cluster in time, often preceded by rapid change bursts). RE-NET’s subgraph RNN directly models “how does the commit neighbourhood evolve over time?” KG-Commit’s TIME and INTERVAL nodes provide structured temporal anchors for all these methods without extra engineering.

Cons. Temporal KGE methods are transductive in time: cannot extrapolate beyond the training time window without retraining or online updates. RE-NET requires a fixed time granularity (daily? per-commit?), and misalignment with KG-Commit’s variable commit rate must be handled. TANGO’s ODE solver is computationally prohibitive at scale. New or small repositories have cold-start problems.

Key papers. TTransE (Leblay & Chekol, WWW 2018), TNTComplex (Lacroix et al., ICLR 2020), RE-NET (Jin et al., ICLR 2020), TiRGN (Li et al., AAAI 2022), TANGO (Han et al., EMNLP 2021).

12 Family 11: Subgraph and Inductive Reasoning

12.1 What It Captures

Inductive KG methods reason over local subgraphs around query entities, enabling generalisation to unseen entities and entirely new graphs (new repositories). GraIL (Teru et al., 2020) extracts the local enclosing subgraph around a query triple $(h, r, ?)$ and applies a GNN to classify whether the triple holds. SNRI (Xu et al., 2022) additionally incorporates node and relation features. NodePiece (Galkin et al., 2022) represents unseen entities by their anchor distances, enabling fully inductive KGE. INDIGO (Liu et al., 2021) uses a differentiable inductive logic system.

Subgraph formulation for KG-Commit.

Query: will COMMIT c touch a FUNCTION f with an existing bug pattern?
Enclosing subgraph: all nodes within $k=2$ hops of (COMMIT_c, FUNCTION_f)
Includes: AUTHOR, FILE, co-changed FILEs, called FUNCTIONs, ISSUES
GraIL GNN: classify (COMMIT_c, INTRODUCES_BUG, ?) on enclosing subgraph
Inductive benefit: new repos at test time not seen during training

Efficiency. GraIL: enclosing subgraph extraction is $O(k^2 \cdot \bar{d})$ for average degree $\bar{d}$ and k hops. GNN over the subgraph: $O(|V_{\text{sub}}| \cdot d^2)$. For $k = 2$ and KG-Commit’s average degree ≈ 10 : subgraphs have $\approx 100\text{--}500$ nodes — fast. Training: similar to GNN baselines. NodePiece: $O(N \cdot K \cdot d)$ for K anchor tokens per entity — highly efficient for large KGs.

Prior Implementations and Difficulty. **GraIL:** PyTorch reference code (Teru et al., ICML 2020), maintained. **SNRI:** PyTorch reference code. **NodePiece:** PyTorch Geometric integration, maintained. **INDIGO:** research-grade. GraIL requires a subgraph extraction function (implementable in NetworkX or Cypher in <1 day). The inductive setting is critical for KG-Commit: the model must generalise to new Apache projects not seen during training. Difficulty: GraIL **moderate**; NodePiece **easy**; SNRI **moderate**.

Pros. Critical for cross-project generalisation: GraIL trained on 5 Apache projects should predict defects in a 6th unseen project without retraining. NodePiece enables deploying a trained model on any new repository. Enclosing subgraphs are interpretable — the extracted subgraph IS the explanation. SNRI’s feature incorporation uses the rich node features from KG-Commit’s construction phase directly.

Cons. Subgraph extraction adds a preprocessing step per query at inference time. For dense KGs (high-degree nodes), enclosing subgraphs can be large and slow to extract. GraIL’s original formulation focuses on link prediction, not node classification — adapting to commit-level prediction requires a task reformulation. INDIGO is too immature for production use.

Key papers. GraIL (Teru et al., ICML 2020), SNRI (Xu et al., EMNLP 2022), NodePiece (Galkin et al., ICLR 2022), INDIGO (Liu et al., NeurIPS 2021).

13 Family 12: LLM-Augmented Knowledge Graph Inference

13.1 What It Captures

LLM-augmented KG methods combine the symbolic structure of a KG with the language understanding and world knowledge of large language models. KnowPrompt (Chen et al., 2022) injects KG entity embeddings into prompt templates for relation classification. Think-on-Graph (Sun et al., 2024) has the LLM interactively walk the KG, selecting which relations to follow. KG-GPT (Kim et al., 2024) prompts GPT-4 with serialised KG subgraphs. KEPLER (Wang et al., 2021) jointly pretrains a BERT-like LM with KGE objectives, producing embeddings that are simultaneously language- and graph-grounded.

Formulation for KG-Commit.

KGLLM prompting approach:

1. Serialise 2-hop neighbourhood of COMMIT_c as text:
"Commit c modified F1, F2. F1 has smell GodClass. F2 was recently fixed for issue #123. Author a has low ownership."
2. LLM reasons: "Is this commit likely to introduce a bug?"
3. Answer grounded by KG facts, not hallucinated

Think-on-Graph:

LLM iteratively selects next KG hop to follow from COMMIT node until sufficient evidence for a prediction is accumulated

KEPLER:

Fine-tune CodeBERT jointly on code MLM + KG triple scoring
Use resulting embeddings as node features for R-GCN (Family 4)

Efficiency. KnowPrompt: fine-tuning cost of a BERT-class model ($\approx 110\text{M}$ params); inference $\approx 100\text{ms}$ per query on GPU. KGLLM/KG-GPT: API inference cost ($\sim \$0.01$ – $\$0.10$ per query with GPT-4, depending on context size). Think-on-Graph: multiple LLM calls per query (iterative KG walking), 3 – $5\times$ cost of single-pass KGLLM. KEPLER pretraining: significant GPU cost (≈ 1 – 4 GPU-days), but inference is standard BERT speed. API-based methods are cost-limited, not compute-limited.

Prior Implementations and Difficulty. **KEPLER:** Hugging Face model checkpoint available. **KnowPrompt:** PyTorch reference code (Chen et al., ACL 2022). **Think-on-Graph:** PyTorch + LLM API (Sun et al., ICLR 2024). KGLLM-style prompting is the fastest to prototype (1 day): write a Cypher query to serialise a commit's neighbourhood as text, pass to Claude/GPT-4. KEPLER fine-tuning on Apache code commits is a medium-term investment. Difficulty: KGLLM prompt **easy**; KnowPrompt **moderate**; KEPLER fine-tuning **moderate**; Think-on-Graph **moderate**.

Pros. LLMs bring *world knowledge* and *natural language reasoning* not available to structural GNNs: they understand what "NPE in concurrent context" implies. KGLLM-style grounding reduces hallucination by anchoring responses to KG facts. Think-on-Graph is highly interpretable — the traversal trace IS the explanation. KEPLER produces code + KG-grounded embeddings that can replace CodeBERT in the GNN node initialisation pipeline.

Cons. API-based LLMs have per-query cost prohibitive for online commit analysis at scale (10^3 – 10^4 commits/day). LLM reasoning over serialised KG text loses graph topology information. KEPLER pretraining is expensive and may need domain-specific retraining. LLM outputs are non-deterministic and hard to calibrate. Context window limits restrict the amount of KG neighbourhood that can be serialised per query.

Key papers. KEPLER (Wang et al., TACL 2021), KnowPrompt (Chen et al., ACL 2022), Think-on-Graph (Sun et al., ICLR 2024), KG-GPT (Kim et al., EACL 2024).

14 Family 13: Contrastive and Self-Supervised Representations

14.1 What It Captures

Self-supervised and contrastive KGE methods reduce dependence on scarce defect labels by learning representations from graph structure alone. SimKGC (Wang et al., 2022) applies contrastive learning with in-batch negative sampling over BERT-encoded entity descriptions. KGE-CL (Yu et al., 2021) applies SimCLR-style augmentations to KG subgraphs (drop edges, mask entities) and trains a GNN with contrastive loss. GraphMAE (Hou et al., 2022) applies masked autoencoding (BERT-style) to graph nodes, masking subsets and reconstructing features. BGRL (Thakoor et al., 2022) is a graph-specific bootstrapped representation learning method requiring no negative samples.

Formulation for KG-Commit.

Self-supervised pre-training (no defect labels needed):

Augmentation 1: drop MODIFIES edges with $p=0.2$

Augmentation 2: mask FUNCTION node features

Contrastive loss: maximise agreement between augmented views

GraphMAE: reconstruct masked FUNCTION features from neighbourhood context

BGRL: bootstrap from online/target network pair -- no negatives needed

Transfer: pre-train on full unlabelled KG, then fine-tune on the small set of defect-labelled commits

Efficiency. SimKGC: BERT-based, ≈ 100 ms per batch inference. KGE-CL / BGRL: GNN-based, $O(|\mathcal{E}| \cdot d^2)$ per epoch with two forward passes (two augmented views). GraphMAE: adds a decoder network ($O(d^2)$ extra parameters). BGRL: single forward pass — the fastest self-supervised GNN; no negatives required. Pre-training on the unlabelled KG: 1–10 hours; fine-tuning on defect labels: minutes.

Prior Implementations and Difficulty. **PyTorch Geometric:** GraphMAE, BGRL, and DGI (Deep Graph Infomax) all available. **SimKGC:** reference code (PyTorch, Wang et al., ACL 2022). For KG-Commit: BGRL is the recommended entry point (simplest, no hyperparameter sensitivity from negative sampling). GraphMAE requires a feature reconstruction decoder. SimKGC requires textual entity descriptions (available from commit messages and Javadoc). Difficulty: BGRL **easy**; GraphMAE **easy**; SimKGC **moderate**; KGE-CL **moderate**.

Pros. Critical given KG-Commit’s label scarcity: most commits are not labelled as buggy or clean. Self-supervised pre-training extracts rich representations from the unlabelled majority. BGRL’s bootstrapped approach avoids the instability of contrastive learning with hard negatives. GraphMAE’s masked-feature pretext task is directly relevant — reconstructing masked code metrics from graph context is a strong pretraining signal for defect prediction.

Cons. Augmentation choices (which edges/features to drop) critically affect quality and must be tuned for KG-Commit’s schema. BGRL may underfit on very heterogeneous graphs (designed primarily for homogeneous graphs). GraphMAE’s reconstruction quality depends on informative node features. Self-supervised pre-training adds a two-stage pipeline (pre-train then fine-tune), increasing total development time. Pre-trained representations may capture structure unrelated to defect prediction.

Key papers. SimKGC (Wang et al., ACL 2022), GraphMAE (Hou et al., KDD 2022), BGRL (Thakoor et al., ICLR 2022), DGI (Veličković et al., ICLR 2019).

15 Family 14: Graph Transformer Architectures

15.1 What It Captures

Graph Transformers apply the self-attention mechanism directly over graph nodes, using structural information as positional/bias terms. Graphormer (Ying et al., 2021) encodes spatial distances between nodes as attention biases and uses degree centrality as node feature augmentation. GPS (Rampášek et al., 2022) combines local MPNN layers with global transformer layers, achieving SOTA on many graph classification benchmarks. The Structure-Aware Transformer (SAT, Chen et al., 2022) extracts a subgraph representation for each node and uses it as the key/value in attention. TokenGT (Kim et al., 2022) treats all nodes and edges as tokens in a sequence, enabling full self-attention without graph-specific inductive bias.

Formulation for KG-Commit.

GPS over KG-Commit subgraph:

Local: R-GCN message passing over typed edges (Family 4)
 Global: transformer self-attention over all nodes in subgraph
 PE: Laplacian eigenvectors + random walk structural encoding
 Output: COMMIT node representation -> defect head (sigmoid)

Graphormer:

Spatial encoding = shortest path distance between all node pairs
 Degree encoding = learnable bias from in/out degree

TokenGT:

Flatten all (node, edge) tokens into a sequence, attend globally

Efficiency. Graphormer/SAT/TokenGT: $O(N^2 \cdot d)$ per layer due to self-attention (quadratic in node count). Practical for subgraphs up to ~ 5000 nodes. For commit-level subgraphs (< 1000 nodes), this is tractable on GPU. GPS: $O(|\mathcal{E}| \cdot d^2 + N^2 \cdot d)$ per layer (MPNN + attention); the attention term dominates for dense subgraphs.

Flash Attention reduces memory but not FLOPs. Laplacian positional encodings add a one-time preprocessing cost per graph.

Prior Implementations and Difficulty. **PyTorch Geometric:** GraphormerConv, GPS layer (via GPSConv), positional encoding transforms. **Graphormer:** reference code (Microsoft Research, production-grade). **GPS:** reference code (Rampášek et al., NeurIPS 2022), maintained. **TokenGT:** reference code (Kim et al., NeurIPS 2022). GPS is the recommended entry point (best benchmark performance, maintained code, modular design combining MPNN + transformer). Difficulty: Graphormer **moderate**; GPS **moderate**; SAT/TokenGT **hard** (compute budget issues at scale).

Pros. Graph Transformers are the current SOTA on many graph classification benchmarks. GPS’s hybrid architecture directly combines the local relational reasoning of R-GCN (Family 5) with the global context of transformers — ideal for KG-Commit where both local code structure and global project-level patterns matter. Graphormer’s spatial encoding captures graph-distance information explicitly, relevant for CPG node distances.

Cons. $O(N^2)$ self-attention is expensive for large graphs. Positional encodings (Laplacian eigenvectors) are sensitive to graph changes and add preprocessing cost. Flash Attention reduces memory but not FLOPs. The performance gains over well-tuned GNNs are modest on many tasks (GPS beats GCN by 3–5% on most benchmarks). Not widely tested on code/KG tasks. SAT and TokenGT are research-grade for large heterogeneous graphs.

Key papers. Graphormer (Ying et al., NeurIPS 2021), SAT (Chen et al., NeurIPS 2022), GPS (Rampášek et al., NeurIPS 2022), TokenGT (Kim et al., NeurIPS 2022).

16 Comprehensive Comparison

Family		Hetero.	Temp.	Induct.	Interp.	Cost	KG-Commit fit
Shallow GCN/-		Partial	–	✓	–	Very low	Baseline
GraphSAGE							
Attention (GAT/HAN)		✓	–	Partial	✓	Low	Good
Gated GNN (GGNN)		Partial	–	–	–	Low	CPG-specific
Relational GCN/HGT)	(R-	✓	–	Partial	–	Medium	Best structural
TransE family		–	–	–	–	Very low	Link pre- diction
Bilinear Mult/ComplEx)	(Dist-	–	–	–	–	Very low	Link pre- diction
Rotation (RotatE/HAKE)	(Ro-	–	–	–	Partial	Low	Best KGE

Family		Hetero.	Temp.	Induct.	Interp.	Cost	KG-Commit fit
Path reasoning (PRA/MIN-ERVA)	✓	–		Partial	✓	Medium	Explainability
Rule learning (AMIE/Neural-LP)	✓	–		✓	✓	Low	Interpretability
Temporal (TTransE/RE-NET)	Partial	✓	–	–		Medium	Temporal signal
Subgraph (GraIL/N-odePiece)	✓	–		✓	✓	Medium	Cross-project
LLM-augmented (KGLLM)	✓	–		✓	✓	High	NL ground-ing
Contrastive (BGRL/Graph-MAE)	Partial	–		✓	–	Medium	Label scarcity
Graph Trans-former (GPS)	✓	–		Partial	–	High	Best accuracy

17 Recommended Phased Inference Plan

1. **Phase 1 — KGE Baseline (Week 1–2).** Export KG-Commit triples and train TransE and RotatE via PyKEEN. Use for link prediction (COMMIT→INTRODUCES_BUG). Establish a strong embedding baseline with zero GNN infrastructure.
2. **Phase 2 — Shallow GNN Baseline.** Build a PyG `HeteroData` object from the KG. Train GraphSAGE (inductive) with code metric node features. This is the JIT-DP baseline against which all structural improvements are measured.
3. **Phase 3 — R-GCN / HGT (Week 4–6).** Add R-GCN with basis decomposition over all edge types. Tune number of layers (2–3), hidden dimension (128–256). Compare against the shallow GNN. This is the primary structural model.
4. **Phase 4 — GGNN on CPG Subgraphs.** For commits with CPG subgraphs (Joern output), apply GGNN as a local feature extractor. Feed its output as a COMMIT node attribute into the R-GCN.
5. **Phase 5 — Self-Supervised Pre-Training.** Pre-train R-GCN or HGT with BGRL or GraphMAE on the unlabelled portion of the KG. Fine-tune on defect labels. Evaluate benefit for label-scarce scenarios.
6. **Phase 6 — Temporal Extension.** Add TNTComplex or RE-NET to incorporate commit sequence information. Evaluate whether temporal signal adds value over structural signal alone.

7. **Phase 7 — Rule Mining.** Run AMIE on the KG triple file. Review mined rules with domain experts. Encode actionable rules as code review guidelines. Integrate high-confidence rules as hard constraints in the GNN.
8. **Phase 8 — Cross-Project / Inductive.** Apply GraIL or NodePiece for inductive evaluation on held-out Apache projects. This evaluates the model’s generalisation beyond the training corpus.
9. **Phase 9 — LLM Grounding (Optional).** Prototype KGLLM-style prompting for a small set of high-confidence defect predictions. Use as an explainability layer on top of the GNN output, not as the primary predictor.
10. **Phase 10 — Graph Transformer.** Replace R-GCN with GPS as the backbone. Compare accuracy vs. training cost. Use only if the accuracy gain justifies the compute overhead.

18 Open Research Questions

- **Heterogeneous GNN for JIT-DP.** No published study directly applies HGT or R-GCN to a multi-typed code KG for defect prediction. This is a core open gap that KG-Commit addresses.
- **Temporal GNN for commit streams.** RE-NET and TiRGN were evaluated on event-based KGs (ICEWS, GDELT) — their transferability to commit-level code KGs is untested.
- **Label-efficient learning.** Can self-supervised pre-training (BGRL/GraphMAE) on the unlabelled KG reduce the labelled-data requirement below the current literature minimum (~ 1000 defective commits)?
- **Rule mining from code KGs.** AMIE has not been applied to commit-level code KGs. What defect-predictive rules can be mined, and are they project-portable?
- **Subgraph reasoning for cross-project transfer.** GraIL’s inductive framework enables zero-shot evaluation on new projects. What is the accuracy gap between in-distribution and cross-project settings for KG-Commit?
- **Composing GNN and KGE.** How to best combine structural GNN representations (R-GCN) with relational KGE representations (RotatE) in a unified model for code-change defect prediction?
- **Explainability for commit review.** Which inference method produces the most actionable explanation for a developer reviewing a flagged commit — paths, rules, attention weights, or LLM traces?

References

- [1] T. Kipf and M. Welling, “Semi-Supervised Classification with Graph Convolutional Networks,” *ICLR*, 2017.
- [2] W. Hamilton et al., “Inductive Representation Learning on Large Graphs,” *NeurIPS*, 2017.

- [3] M. Defferrard et al., “Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering,” *NeurIPS*, 2016.
- [4] F. Wu et al., “Simplifying Graph Convolutional Networks,” *ICML*, 2019.
- [5] P. Veličković et al., “Graph Attention Networks,” *ICLR*, 2018.
- [6] S. Brody et al., “How Attentive are Graph Attention Networks?” *ICLR*, 2022.
- [7] X. Wang et al., “Heterogeneous Graph Attention Network,” *WWW*, 2019.
- [8] Y. Li et al., “Gated Graph Sequence Neural Networks,” *ICLR*, 2016.
- [9] Z. Zhou et al., “Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks,” *NeurIPS*, 2019.
- [10] K. Tai et al., “Improved Semantic Representations from Tree-Structured Long Short-Term Memory Networks,” *ACL*, 2015.
- [11] M. Schlichtkrull et al., “Modeling Relational Data with Graph Convolutional Networks,” *ESWC*, 2018.
- [12] Z. Hu et al., “Heterogeneous Graph Transformer,” *WWW*, 2020.
- [13] S. Vashishth et al., “Composition-based Multi-Relational Graph Convolutional Networks,” *ICLR*, 2020.
- [14] A. Bordes et al., “Translating Embeddings for Modeling Multi-relational Data,” *NeurIPS*, 2013.
- [15] Z. Wang et al., “Knowledge Graph Embedding by Translating on Hyperplanes,” *AAAI*, 2014.
- [16] Y. Lin et al., “Learning Entity and Relation Embeddings for Knowledge Graph Completion,” *AAAI*, 2015.
- [17] B. Yang et al., “Embedding Entities and Relations for Learning and Inference in Knowledge Bases,” *ICLR*, 2015.
- [18] T. Trouillon et al., “Complex Embeddings for Simple Link Prediction,” *ICML*, 2016.
- [19] I. Balažević et al., “TuckER: Tensor Factorization for Knowledge Graph Completion,” *EMNLP*, 2019.
- [20] M. Nickel et al., “A Three-Way Model for Collective Learning on Multi-Relational Data,” *ICML*, 2011.
- [21] Z. Sun et al., “RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space,” *ICLR*, 2019.
- [22] Z. Zhang et al., “Learning Hierarchy-Aware Knowledge Graph Embeddings for Link Prediction,” *ACL*, 2020.
- [23] L. Chao et al., “PairRE: Knowledge Graph Embeddings via Paired Relation Vectors,” *ACL*, 2021.
- [24] S. Zhang et al., “Quaternion Knowledge Graph Embeddings,” *NeurIPS*, 2019.

- [25] N. Lao et al., “Random Walk Inference and Learning in a Large Scale Knowledge Base,” *EMNLP*, 2011.
- [26] W. Xiong et al., “DeepPath: A Reinforcement Learning Method for Knowledge Graph Reasoning,” *EMNLP*, 2017.
- [27] R. Das et al., “Go for a Walk and Arrive at the Answer: Reasoning Over Paths in Knowledge Bases,” *ICLR*, 2018.
- [28] Y. Shen et al., “M-Walk: Learning to Walk over Graphs using Monte Carlo Tree Search,” *NeurIPS*, 2018.
- [29] L. Galárraga et al., “AMIE: Association Rule Mining under Incomplete Evidence in Ontological Knowledge Bases,” *WWW*, 2013.
- [30] F. Yang et al., “Differentiable Learning of Logical Rules for Knowledge Base Reasoning,” *NeurIPS*, 2017.
- [31] A. Sadeghian et al., “DRUM: End-To-End Differentiable Rule Mining on Knowledge Graphs,” *NeurIPS*, 2019.
- [32] M. Qu et al., “RNNLogic: Learning Logic Rules for Reasoning on Knowledge Graphs,” *ICLR*, 2021.
- [33] J. Leblay and M. Chekol, “Deriving Validity Time in Knowledge Graph,” *WWW*, 2018.
- [34] T. Lacroix et al., “Tensor Decompositions for Temporal Knowledge Base Completion,” *ICLR*, 2020.
- [35] W. Jin et al., “Recurrent Event Network: Autoregressive Structure Inference over Temporal Knowledge Graphs,” *ICLR*, 2020.
- [36] Z. Li et al., “TiRGN: Time-Guided Recurrent Graph Network with Local-Global Historical Patterns for Temporal Knowledge Graph Reasoning,” *AAAI*, 2022.
- [37] Z. Han et al., “TANGO: Time-Reversal Latent GraphODE for Multi-Relational Temporal Knowledge Graph Completion,” *EMNLP*, 2021.
- [38] K. Teru et al., “Inductive Relation Prediction by Subgraph Reasoning,” *ICML*, 2020.
- [39] M. Xu et al., “Subgraph Neighboring Relations Infomax for Inductive Link Prediction on Knowledge Graphs,” *EMNLP*, 2022.
- [40] M. Galkin et al., “NodePiece: Compositional and Parameter-Efficient Representations of Large Knowledge Graphs,” *ICLR*, 2022.
- [41] Z. Liu et al., “INDIGO: GNN-Based Inductive Knowledge Graph Completion Using Pair-Wise Encoding,” *NeurIPS*, 2021.
- [42] X. Wang et al., “KEPLER: A Unified Model for Knowledge Embedding and Pre-trained Language Representation,” *TACL*, 2021.
- [43] Z. Chen et al., “KnowPrompt: Knowledge-aware Prompt-tuning with Synergistic Optimization for Relation Extraction,” *ACL*, 2022.
- [44] J. Sun et al., “Think-on-Graph: Deep and Responsible Reasoning of LLMs with Knowledge Graph,” *ICLR*, 2024.

- [45] W. Kim et al., “KG-GPT: A General Framework for Reasoning on Knowledge Graphs Using Large Language Models,” *EACL*, 2024.
- [46] W. Wang et al., “SimKGC: Simple Contrastive Knowledge Graph Completion with Pre-trained Language Models,” *ACL*, 2022.
- [47] Z. Hou et al., “GraphMAE: Self-Supervised Masked Graph Autoencoders,” *KDD*, 2022.
- [48] S. Thakoor et al., “Large-Scale Representation Learning on Graphs via Bootstrapping,” *ICLR*, 2022.
- [49] P. Veličković et al., “Deep Graph Infomax,” *ICLR*, 2019.
- [50] C. Ying et al., “Do Transformers Really Perform Bad for Graph Representation?” *NeurIPS*, 2021.
- [51] D. Chen et al., “Structure-Aware Transformer for Graph Representation Learning,” *NeurIPS*, 2022.
- [52] L. Rampásek et al., “Recipe for a General, Powerful, Scalable Graph Transformer,” *NeurIPS*, 2022.
- [53] J. Kim et al., “Pure Transformers are Powerful Graph Learners (TokenGT),” *NeurIPS*, 2022.
- [54] M. Ali et al., “PyKEEN 1.0: A Python Library for Training and Evaluating Knowledge Graph Embeddings,” *JMLR*, 2021.